

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Agustín Adolfo Gómez Ríos

11 de septiembre de 2026

00:53

1. Introducción

En el presente informe se expone el análisis de dos problemas de la computación muy populares; la multiplicación de matrices y el ordenamiento de arreglos unidimensionales. Buscamos comparar la complejidad asintótica teórica de los diversos algoritmos, para los ordenamientos analizaremos MERGE SORT [3], QUICK SORT [5], PATIENCE SORT [4] y `std::sort`, mientras que para la multiplicación de matrices veremos los algoritmos de NAIVE y STRASSEN [6]. Evaluaremos el comportamiento de ambos problemas ante altos volúmenes de datos a través de distintas distribuciones, tamaños de entrada y dominios. Con esto buscamos evidenciar cómo los factores del hardware impactan en los tiempos de ejecución, demostrando en la práctica las limitaciones de depender exclusivamente de la notación Big O

2. Implementaciones

El código fuente en C++, los scripts de Python utilizados para la generación de datos y gráficos, y todos los archivos de salida de este estudio experimental se encuentran estructurados y disponibles en el siguiente repositorio de GitHub

<https://github.com/agomezrr/Tarea1Agustin>

3. Entorno experimental

Para medir los tiempos de forma justa y poder replicar las pruebas sin problemas, aislamos las funciones de cada algoritmo usando la librería `<chrono>` de C++, midiendo todo en microsegundos para no perder precisión en los tamaños más chicos.

Detalles del entorno y hardware:

- **Sistema Operativo:** WSL 2, subsistema Ubuntu en Windows 11.
- **Hardware:** Procesador Intel Core i5-10400F con 24 GB de memoria RAM.
- **Compilación:** Todo compilado a través de `Makefile` usando `g++`
- **Procesamiento y gráficos:** Se programaron scripts en Python con `pandas`, `matplotlib` y `seaborn` para crear las entradas de prueba y generar las graficas.

Casos de prueba analizados:

- **Ordenamiento de Arreglos:** Probamos tamaños de $N \in \{10, 10^3, 10^5, 10^7\}$ en orden ascendente, descendente y aleatorio, combinados con los dominios $D1$ y $D7$.
- **Matrices:** Probamos dimensiones de $N \in \{16, 64, 256, 1024\}$ para matrices densas, dispersas y diagonales, usando valores en los dominios $D0$ y $D10$.

4. Resultados y Análisis

4.1. Algoritmos de Ordenamiento

Al probar los algoritmos de ordenamiento, se ven grandes diferencias en la práctica, a pesar de que en teoría todos operan en $O(n \log n)$. Para los algoritmos podemos comentar que para;

1. Merge Sort: Para casi todas las pruebas, Merge Sort terminó siendo el más lento (tardando más de 1200 ms con 10^7 datos aleatorios). El problema no son las comparaciones, sino que es un algoritmo del tipo *out-of-place*, es decir, requiere memoria adicional proporcional al tamaño del arreglo original $O(n)$ para poder realizar el sort. Al estar creando vectores temporales (`std::vector`) y copiando datos a cada rato, el procesador se satura manejando la memoria.

2. Quick Sort: Durante las primeras pruebas, la implementación clásica de Quick Sort [5] colapsó, como teníamos que elegir siempre el último elemento como pivote y usar partición simple, el algoritmo caía en su peor caso ($O(n^2)$) frente a arreglos ya ordenados o con muchos datos repetidos (Dominio D1), tardando una eternidad. Tras solucionar esto cambiando el pivote al centro e implementando la partición de 3 vías (llamado *Dutch National Flag* en internet [1]), Quick Sort y `std::sort` no tuvieron más problemas y funcionaron excelente. Como son algoritmos *in-place* y solo hacen intercambios (swap) en la misma memoria, aprovechan súper bien la memoria caché. Llamó mucho la atención lo rápido que fue QUICK SORT en el Dominio D1, lo que confirma que el *Dutch National Flag* fue un éxito total, ya que agrupa los repetidos, los ignora para la siguiente recursión y achica bastante el árbol recursivo.

3. Patience Sort: Después de optimizarlo con búsqueda binaria (`std::lower_bound`) y usando un Min-Heap para la cola de prioridad, el Patience Sort logró un crecimiento súper estable. Si bien es un poco más lento que los algoritmos *in-place* por todo el trabajo de mantener el stack y el heap, demostró ser una opción muy robusta para procesar millones de datos sin ningún problema.

4. Análisis general y de memoria: Al analizar las curvas, se observa un disparo abrupto en los tiempos a partir de $N = 10^5$. Este comportamiento obedece a dos factores críticos. En primer lugar, la escala de las gráficas condensa la inyección de 9.900.000 datos exclusivamente en el último tramo, haciendo que el salto asintótico se vea visualmente desproporcionado. En segundo lugar, existe un límite físico del hardware, un arreglo de 10^5 enteros (aproximadamente 400 KB) cabe holgadamente en la memoria rápida del procesador (como los 12 MB de caché L3 que tiene el Intel Core i5-10400F donde se ejecutaron las pruebas). Operar aquí garantiza latencias mínimas. Sin embargo, al procesar 10^7 elementos, la estructura pasa a pesar 40 MB, rebasando por completo la capacidad de la caché y obligando a la CPU a buscar datos en la RAM. Esto explica por qué el rendimiento de MERGE SORT se degrada tan severamente en el último tramo: al ser out-of-place, crea vectores temporales que duplican la memoria a casi 80 MB, saturando el bus de datos hacia la RAM mucho más rápido que las alternativas in-place.

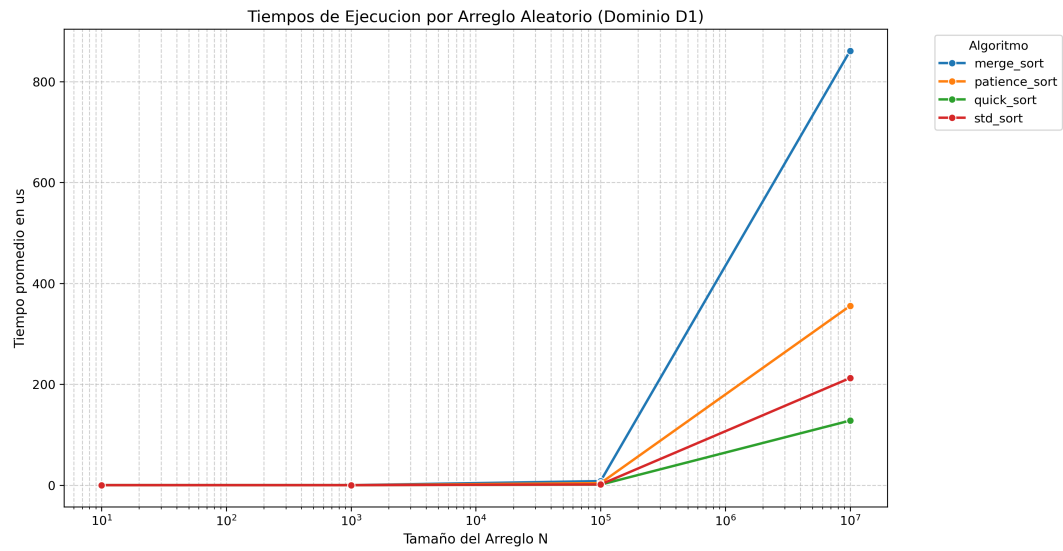


Figura 1: Arreglo Aleatorio (D1)

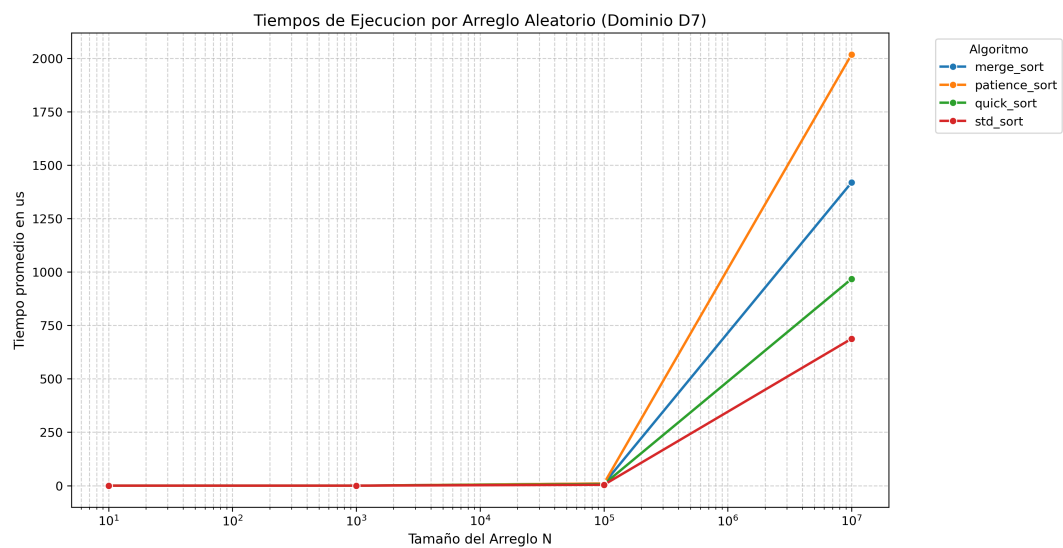


Figura 2: Arreglo Aleatorio (D7)

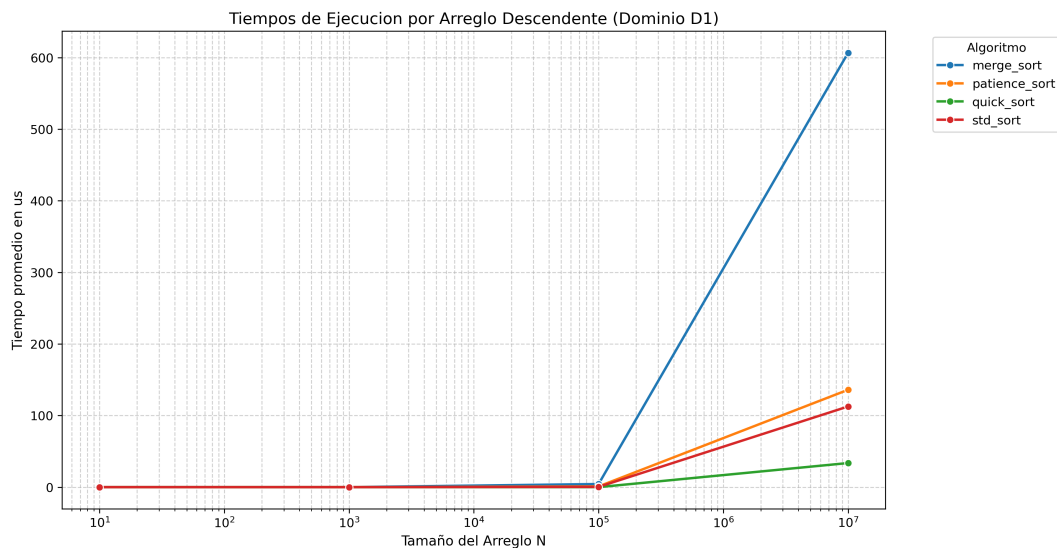


Figura 3: Arreglo Descendente (D1)

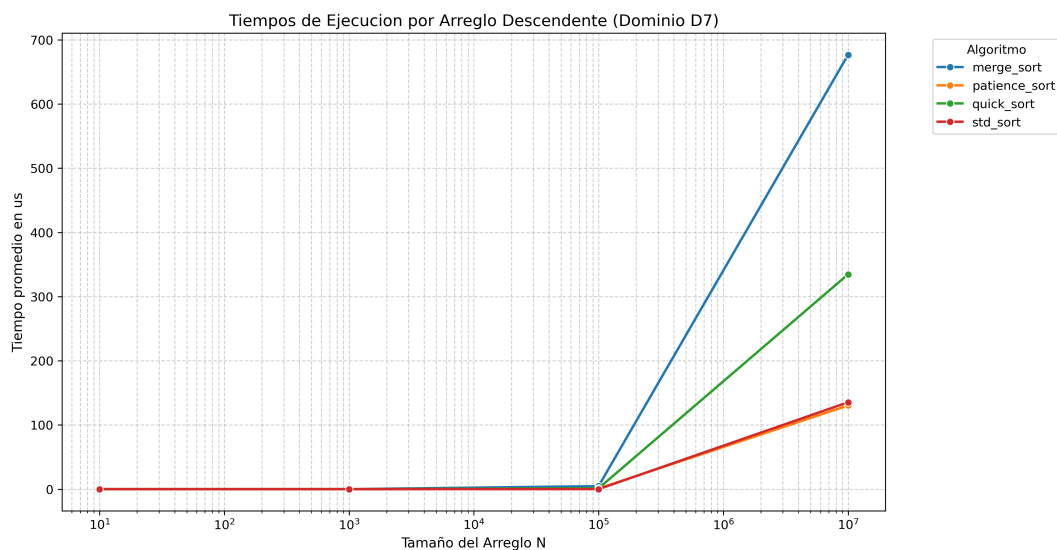


Figura 4: Arreglo Descendente (D7)

Se omitieron las gráficas del arreglo Ascendente ya que su comportamiento asintótico no presenta conclusiones adicionales de interés respecto a los casos presentados.

4.2. Multiplicación de Matrices

Pasando a la multiplicación de matrices, encontramos con dos cosas muy interesantes en la práctica que van un poco en contra de lo que esperábamos en teoría.

1. No importan los 0s: Los tiempos fueron prácticamente idénticos sin importar si la matriz era densa, dispersa o diagonal. Esto pasa porque usamos `std::vector`, que no están optimizados para comprimir o ignorar los ceros, así que nuestro programa igual multiplica todo coordenada por coordenada haciendo

el mismo esfuerzo.

2. Naïve le gana a Strassen: Aquí vimos que el algoritmo Naïve ($O(n^3)$) le ganó con creces a Strassen ($O(n^{2.81})$, demostración del valor en la cita adyacente) [6] en todos los tamaños probados (incluso hasta $N = 1024$). Aunque la teoría dice que Strassen es mejor [2], en la práctica sufre mucho creando submatrices y pidiendo memoria nueva en cada paso recursivo. Naïve, al tener tres simples ciclos for, aprovecha mucho mejor la memoria caché y termina siendo bastante más rápido para estos tamaños.

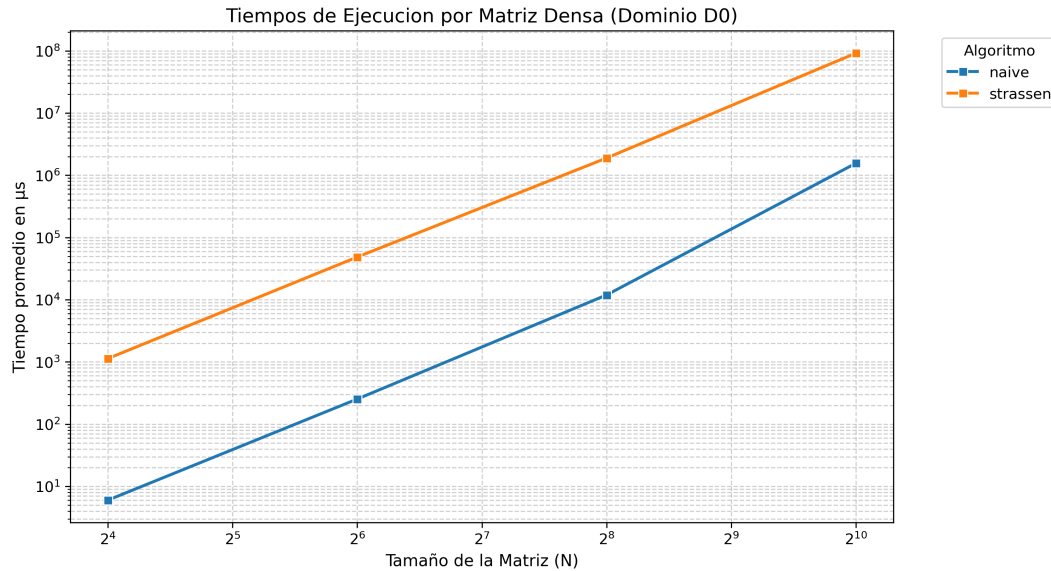


Figura 5: Matriz Densa (D0)

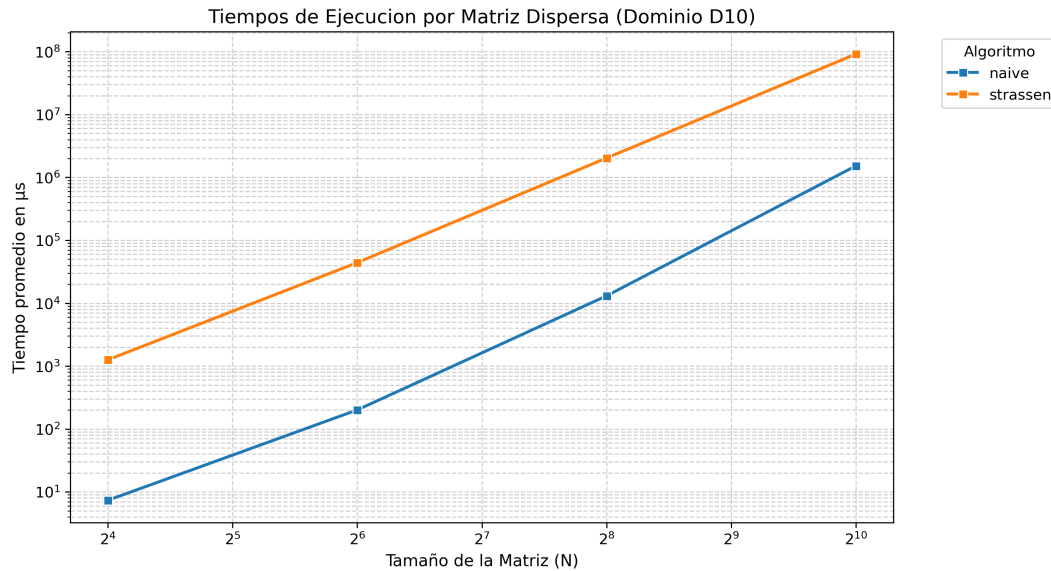


Figura 6: Matriz Dispersa (D10)

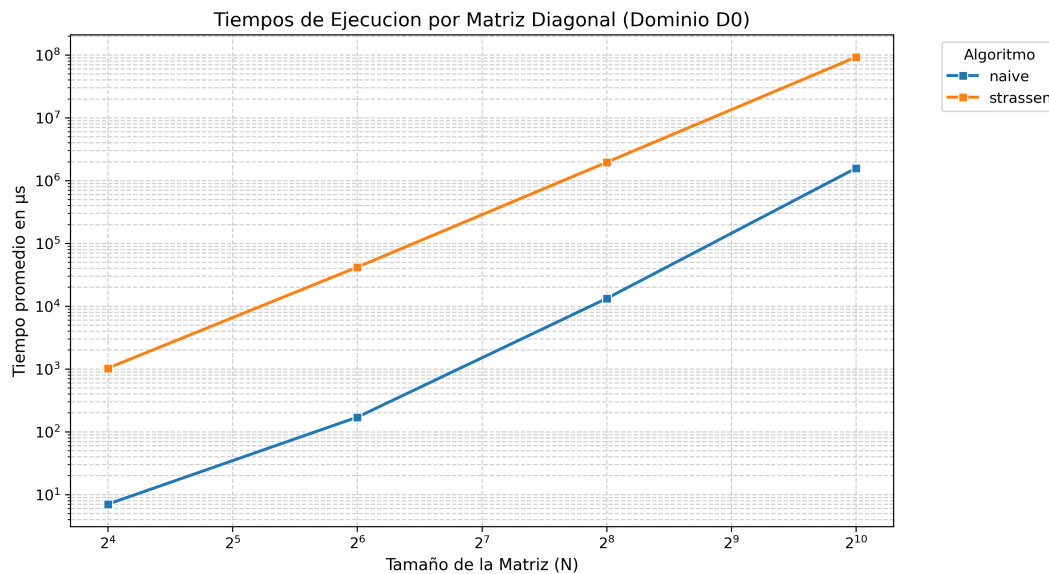


Figura 7: Matriz Diagonal (D0)

5. Conclusiones

En conclusión, las pruebas nos demostraron que la notación teórica Big-O no cuenta toda la historia a la hora de predecir el rendimiento real. Factores físicos del hardware, como aprovechar bien la memoria caché o el alto costo de estar pidiendo memoria dinámica, cambian drásticamente los tiempos de ejecución. Esto lo vimos clarísimo cuando los algoritmos *in-place* como Quick Sort y `std::sort` estuvieron por encima de Merge Sort, o cuando Naive le ganó con creces a Strassen simplemente porque este último satura al procesador creando submatrices en cada paso recursivo. En resumen, para desarrollar un buen software no basta con que la matemática del algoritmo sea eficiente, tenemos que programarlo pensando en que la máquina maneja sus recursos por dentro.

Referencias

- [1] GeeksforGeeks. 3-Way QuickSort (Dutch National Flag). <https://www.geeksforgeeks.org/dsa/3-way-quicksort-dutch-national-flag/>. Accedido: 23 de agosto de 2026. 2023.
- [2] GeeksforGeeks. Matrix Multiplication. <https://www.geeksforgeeks.org/dsa/strassens-matrix-multiplication/>. Accedido: 25 de agosto de 2026. 2025.
- [3] GeeksforGeeks. Merge Sort Algorithm. <https://www.geeksforgeeks.org/dsa/merge-sort/>. Accedido: 21 de agosto de 2026. 2024.
- [4] GeeksforGeeks. Patience Sorting. <https://www.geeksforgeeks.org/dsa/patience-sorting/>. Accedido: 22 de agosto de 2026. 2024.
- [5] GeeksforGeeks. Quick Sort Algorithm. <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>. Accedido: 22 de agosto de 2026. 2024.

- [6] The Startup / Medium. *Strassen's Matrix Multiplication Algorithm*. <https://medium.com/swlh/strassens-matrix-multiplication-algorithm-936f42c2b344>. Accedido: 25 de agosto de 2026. 2020.